

La Persistenza degli oggetti e la loro serializzazione in Java

Breve panoramica

Persistenza degli Oggetti in Informatica

La persistenza degli oggetti consente di mantenere lo stato di un oggetto oltre il ciclo di vita del programma, salvandolo in memoria non volatile (disco, database). È fondamentale in applicazioni come sistemi bancari, gestionali o web.

Esempi d'Uso

- Serializzazione in Java: Oggetti che implementano Serializable possono essere salvati su file tramite ObjectOutputStream.
Esempio: un oggetto ContoCorrente viene serializzato e ripristinato con ObjectInputStream.
- JPA (Java Persistence API): Mappa oggetti Java a tabelle di database. Usato in applicazioni enterprise per salvare dati utente, sessioni, ecc.
- Applicazioni desktop: Framework come Berkeley DB Java Edition permettono persistenza locale efficiente.

Esempi di Non Uso

- Dati volatili: Oggetti legati a risorse di sistema (es. descrittori di file, connessioni di rete) non devono essere serializzati perché perdono significato al riavvio.
- Oggetti temporanei: Come iteratori o thread, che rappresentano stati transitori.
- Classi di sistema: Ad esempio, Font in Java AWT contiene riferimenti al sistema operativo e non è serializzabile.

Errori da Evitare

- Serializzare dati sensibili senza protezione (es. password, chiavi).
- Ignorare serialVersionUID: Se non definito esplicitamente, Java ne genera uno automaticamente. Cambiamenti nella classe possono causare InvalidClassException.
- Salvare riferimenti ciclici senza controlli, rischiando stack overflow.
- Deserializzare da fonti non attendibili senza validazione (Remote Code Execution).
 - Iniezione di payload: Strumenti come ysoserial generano oggetti serializzati che sfruttano classi presenti nel classpath (es. TemplatesImpl di JDK) per eseguire comandi.
 - Accesso a classi interne: Se un oggetto serializzato carica una classe non pubblica, può compromettere il sistema.
- Attacchi XXE o RCE: Combinando serializzazione con parser XML vulnerabili.

Serializzazione e Deserializzazione

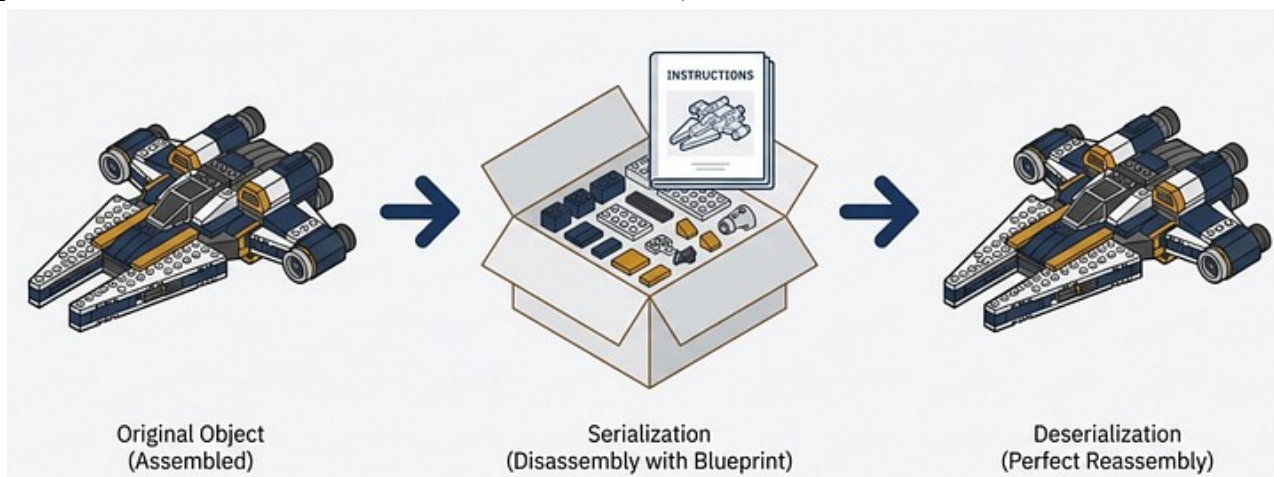
Come già accennato, **serializzazione** in informatica è il processo di conversione dello stato di un oggetto in un formato che può essere salvato in modo permanente su un dispositivo di memoria secondario (ad esempio un HDD/SSD o disco ottico) o trasmesso ad altri dispositivi connessi in rete (ad esempio il PC di un collega o il server). Nasce quindi per consentire la persistenza degli oggetti, ovvero il salvataggio del loro stato per poterli ripristinare in un momento successivo, anche dopo la chiusura del programma.

Il processo inverso (ripristino in memoria principale durante l'esecuzione del programma) si chiama **deserializzazione**, che ricostruisce l'oggetto da un flusso di dati (che sia in locale o da una rete), ripristinando esattamente lo stato originale.

Esempio di vita "quotidiana"

Immagina di aver costruito un intricato modellino con i LEGO. Pubblichi la foto su un social network e un tuo contatto ti chiede se puoi mandargliene una copia. Hai due scelte:

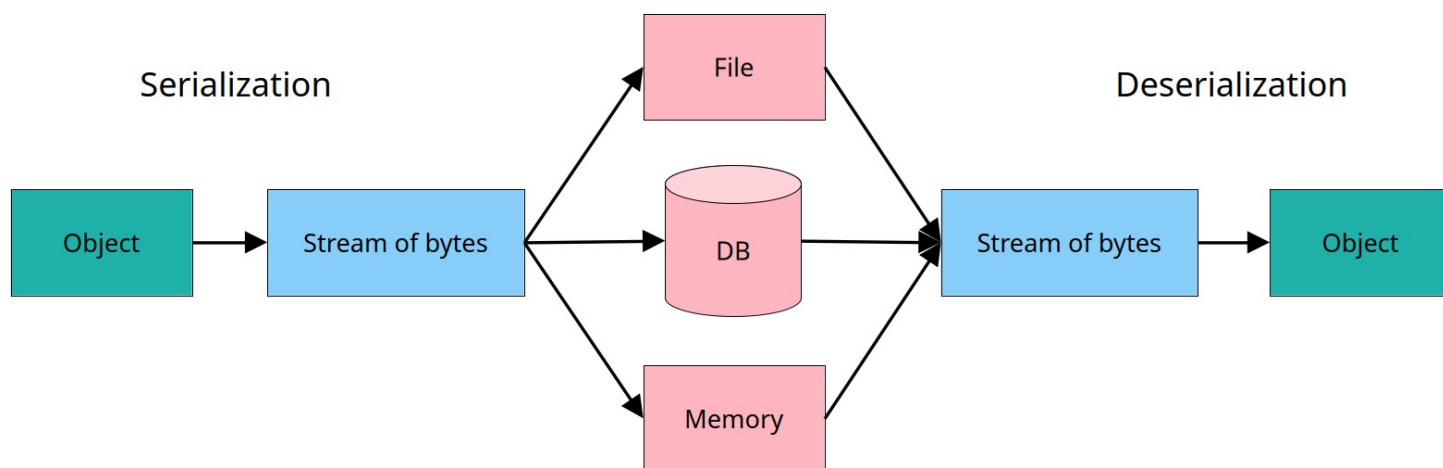
- Mandargli l'oggetto per intero, sperando che il servizio postale abbia cura di non usare i pacchi per una partita estemporanea di calcetto;
- Prendere tutti i pezzi necessari, sistamarli in una scatola e allegare un libretto di istruzioni che permetta al tuo contatto di poter gioire della tua stessa soddisfazione nell'assemblare il modellino con le proprie mani (praticamente il modello di business di IKEA e LEGO).



Perché serializzare un oggetto, scenari d'uso:

- **Comunicazione:** se due macchine eseguono lo stesso codice e devono comunicare, un modo semplice è che una macchina costruisca un oggetto con le informazioni che desidera trasmettere e poi serializzi tale oggetto sull'altra macchina. Non è il metodo migliore per la comunicazione, ma svolge il suo compito.
- **Persistenza:** se si desidera memorizzare lo stato di una particolare operazione in un database, è possibile serializzarlo facilmente in un array di byte e memorizzarlo nel database per un successivo recupero.
- **Copia approfondita:** se si necessita di una replica esatta di un oggetto e non si vuole prendersi la briga di scrivere una propria classe clone() specializzata, è sufficiente serializzare l'oggetto in un array di byte e poi deserializzarlo in un altro oggetto.
- **Memorizzazione nella cache:** in realtà è solo un'applicazione di quanto detto sopra, ma a volte un oggetto impiega 10 minuti per essere creato, mentre ne bastano solo 10 secondi per essere deserializzato. Quindi, invece di conservare l'oggetto gigante in memoria, è sufficiente memorizzarlo in un file tramite serializzazione e leggerlo in seguito quando necessario.
- **Sincronizzazione tra JVM:** la serializzazione funziona su diverse Java Virtual Machine che potrebbero essere in esecuzione su architetture diverse.

Come serializzare un oggetto



Sebbene a livello macchina, tutto è identificabile come un flusso di dati (il file binario del tuo videogioco preferito, così come la ricetta per le orecchiette con le cime di rapa di nonna Rosa scritta sulle note del cellulare), si tende a identificare due tipi di serializzazione:

- **Binaria:** converte gli oggetti in un flusso di byte in formato binario, che non è leggibile dall'uomo ma è molto efficiente in termini di spazio e velocità. È ideale per memorizzazione su file o trasmissione su rete.
- **Testuale:** converte gli oggetti in un formato testuale leggibile sia dall'uomo che dal computer, facilitando il debug, l'interoperabilità e la condivisione tra sistemi diversi. I formati più diffusi sono JSON, XML e YAML. Questo tipo di serializzazione è meno efficiente in termini di dimensione e prestazioni rispetto al binario, ma è più facile da interpretare e modificare manualmente. Ad esempio, JSON è ampiamente usato in API web e configurazioni di applicazioni.

Operazioni preliminari: cosa si può serializzare e perché.

A livello informatico, i pezzi LEGO sono gli attributi degli oggetti istanziati durante l'esecuzione del programma, non i metodi o le costanti già inizializzate nella classe di riferimento e nemmeno le variabili thread, InputStream / OutputStream, Lock / Semaphore, Socket, Connection, Logger perché rappresentano uno stato legato alla JVM in esecuzione, che non ha senso o non è possibile riprodurre altrove. Questo vuol dire che non tutto è serializzabile e che ci sono cose che è meglio non serializzare (come lo scontrino con su il prezzo, se il modellino LEGO è un regalo).

Per fare ciò il computer, in fase di “riassembaggio” dei dati, richiede non solo la conoscenza della classe dell'oggetto da ripristinare, ma anche delle sue sottoclassi e di tutte le classi e relative sottoclassi di tutti gli oggetti contenuti nello stesso, e così via in modo ricorsivo.

Infatti tutti gli attributi delle sottoclassi della classe dell'oggetto e tutti gli attributi delle classi degli oggetti contenuti in esso e delle relative sottoclassi devono essere a loro volta serializzati, e così via, fino ad arrivare a oggetti che contengono solo tipi primitivi.

La scansione ricorsiva di cui sopra deve essere in grado di riconoscere le dipendenze cicliche (ad esempio l'oggetto A contiene un riferimento a un oggetto B che a sua volta contiene un riferimento ad A), altrimenti il programma entrerebbe in un loop infinito.

Nota bene: serializzare una classe derivata da un'altra superclasse, non obbliga la serializzazione di quest'ultima. Tuttavia, ci sono due punti importanti da considerare:

- I campi ereditati dalla superclasse non serializzabile non vengono salvati durante la serializzazione.
- La superclasse deve avere un costruttore senza argomenti (default), perché viene richiamato durante la deserializzazione per ricostruire la parte dell'oggetto relativa alla superclasse.

Serializzazione e Deserializzazione Binaria in Java

Java di suo è munito di strumenti appositi per trasporre in formato binario i dati da serializzare.

L'interfaccia *Serializable*

Serializable è un'interfaccia marker (senza metodi) del pacchetto **java.io**, che indica che una classe può essere serializzata in un flusso di byte. Non richiede l'implementazione di metodi: **basta dichiarare `implements Serializable`**.

```
import java.io.Serializable;
public class Persona implements Serializable {
    // codice della classe
}
```

Possiamo riassumere quanto detto fin'ora dicendo che possono essere serializzati solo oggetti di classi serializzabili.

Una classe è serializzabile se:

- implementa (implements) l'interfaccia *Serializable*
- deriva (extend) da una classe serializzabile

Nota bene: La serializzazione non memorizza gli attributi statici di un oggetto. Nella deserializzazione a tali attributi viene assegnato il valore di default.

Una classe serializzabile deve

- derivare (mediante la relazione gerarchica) da e/o “contenere” solo classi serializzabili
- derivare da e/o “contenere” classi non serializzabili purché ciascuna di queste ultime abbia definito un costruttore senza argomenti e accessibile

Nota bene: Durante la deserializzazione gli oggetti delle classi non serializzabili vengono creati usando il costruttore senza argomenti (senza fare altro)

Serializzazione e Deserializzazione for Dummies

La serializzazione delle istanze di una classe viene gestita dal metodo **defaultWriteObject** della classe **ObjectOutputStream**.

Deve essere chiamato solo all'interno di un metodo **`private void writeObject (ObjectOutputStream output)`**

Questo metodo scrive automaticamente tutto ciò che è richiesto per ricostruire le istanze di una classe, e cioè

- la classe dell'oggetto;
- la firma della classe;
- I valori di tutte le variabili che non siano transient o static.

A questo punto la Deserializzazione avviene in modo speculare, tramite il metodo **defaultReadObject** della classe **ObjectInputStream**. In questa fase non viene attivato il costruttore perché l'oggetto è già virtualmente inizializzato.

Deve essere chiamato solo all'interno di un metodo **`private void readObject (ObjectInputStream input)`** personalizzato per deserializzare automaticamente i campi non static e non transient della classe corrente.

Rispettando sempre la regola per cui **i metodi default...()** vengono sempre eseguiti per primi, faccio presente l'esistenza di metodi speculari, della tipologia `writeTIPO(TIPO)` e `readTIPO(TIPO)`, appartenenti alle classi **ObjectOutputStream** e **ObjectInputStream**, che si possono usare solo durante la serializzazione/deserializzazione, tenendo bene in conto l'ordine di esecuzione: ad esempio se `writeUTF(String)` avviene alla seconda scrittura dopo `defaultWriteObject()`, anche `readUTF(String)` deve avvenire alla seconda lettura dopo `defaultReadObject()`.

Per molte classi questo comportamento è soddisfacente. A volte, tuttavia, il programmatore può volere un maggior controllo su questo meccanismo. Si può personalizzare la serializzazione per le proprie classi fornendo due metodi d'istanza `writeObject` e `readObject` scritti per le specifiche esigenze, ma sempre in modo speculare.

```

import java.io.Serializable;
import java.io.FileInputStream; // in questo esempio viene utilizzato un FILE_PATH, cioè uno String, per definire
import java.io.FileOutputStream; // il file da lavorare, ma è possibile passare direttamente un Oggetto File
import java.io.FileNotFoundException;
import java.io.ObjectOutputStream;
import java.io.ObjectInputStream;
import java.io.IOException;

class Utente implements Serializable { // in questo esempio è volutamente omessa la serialVersionUID
    private String nome, email;
    private transient String password; // ignorata: sarà null dopo deserializzazione

    public Utente (String nome, String email, String password) {
        this.nome = nome;
        this.email = email;
        this.password = password;
    }
    @Override
    public String toString() {
        return String.format("Utente { nome='%s', email='%s', password='%s'}", nome, email, password);
    }
    // Metodo pubblico chiamabile dall'esterno per salvare
    public void salva() throws IOException, FileNotFoundException { // se il file non esiste, lo crea
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE_PATH))) {
            oos.writeObject(this); // utilizzando try-with-resources → close() automatico
        } // ← qui viene chiamato oos.close() automaticamente
    }
    // Metodo statico pubblico chiamabile dall'esterno per caricare
    public static Utente carica() throws IOException, ClassNotFoundException, FileNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(FILE_PATH))) {
            return (Utente) ois.readObject(); // notare il casting a Utente perché readObject() restituisce un Object
        } // ← qui viene chiamato ois.close() automaticamente
    }
    // I due metodi seguenti vengono chiamati automaticamente dalla JVM durante l'esecuzione
    // oos.writeObject(this) e ois.readObject() dei due metodi salva e carica
    private void writeObject (ObjectOutputStream outputFileStream) throws IOException {
        outputFileStream.defaultWriteObject(); // Salva i campi standard
    }
    private void readObject (ObjectInputStream inputFileStream) throws IOException, ClassNotFoundException {
        inputFileStream.defaultReadObject(); // Ripristina i campi standard
    }
}

public class Main {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        // All'avvio: salva
        Utente originale = new Utente("Mario Rossi", "mario@example.com", "s3cr3t!");
        IO.println("=== PRIMA ===\n" + originale);
        originale.salva(); // ← chiamata diretta dalla classe
        IO.println("\n>>> Salvato su file");
        // All'avvio: carica
        Utente ripristinato = Utente.carica(); // ← chiamata diretta dalla classe
        IO.println("\n=== DOPO ===\n" + ripristinato);
    }
}

```

```

**Output:**
=== PRIMA ===
Utente { nome='Mario Rossi', email='mario@example.com', password='s3cr3t!' }

>>> Salvato su file

=== DOPO ===
Utente { nome='Mario Rossi', email='mario@example.com', null }

```

Nota bene anche a costo di ripetersi: `defaultReadObject` e `defaultWriteObject` non sono metodi pubblici chiamabili liberamente; appartengono rispettivamente a `ObjectInputStream` e `ObjectOutputStream` e richiedono un contesto attivo di serializzazione per funzionare. Non restituiscono nemmeno un oggetto: `defaultReadObject` è `void`.

La JVM riconosce `writeObject` e `readObject` solo con una firma precisa, che deve essere:

- ➔ `private void writeObject(ObjectOutputStream oos) throws IOException`
- ➔ `private void readObject(ObjectInputStream ois) throws IOException, ClassNotFoundException`

Se cambi la firma (togliendo il parametro, cambiando il tipo di ritorno, ecc.) la JVM semplicemente li ignora e usa la serializzazione di default, oppure lancia un errore.

Serializzazione fatta seriamente: le classi `ObjectOutputStream` e `ObjectInputStream`

Come si è potuto ben intendere su quanto detto sin ora, le classi `ObjectInputStream` e `ObjectOutputStream` definiscono streams (basati su streams di byte) su cui si possono leggere e scrivere oggetti.

ObjectOutputStream	ObjectInputStream
Si inizializza tramite l'utilizzo di try – catch – finally o try-with-resources	Si inizializza tramite l'utilizzo di try – catch – finally o try-with-resources
<u>Si costruisce passando un <code>OutputStream</code></u> , ad esempio FileOutputStream	<u>Si costruisce passando un <code>InputStream</code></u> , ad esempio FileInputStream
Il metodo principale è writeObject(Object obj)	Il metodo principale è readObject()
Supporta anche metodi come writeInt(int) , writeUTF(String) , writeBoolean(Boolean) per tipi primitivi	Supporta anche metodi come readInt() , readUTF() , readBoolean() per leggere tipi primitivi
Durante la scrittura di un oggetto, vengono inclusi tutti gli oggetti raggiungibili (grafo), mantenendo le relazioni	Durante la lettura, ricostruisce l'intero grafo degli oggetti serializzati, mantenendo le relazioni.
Usa defaultWriteObject() e writeStreamHeader() per gestire internamente la struttura del flusso.	Usa defaultReadObject() e readStreamHeader() per ripristinare i campi non transient e non static della classe.
Deve essere chiuso con close() per liberare le risorse.	Deve essere chiuso con close() per liberare le risorse.

Come si può notare, le due classi sono state costruite per essere speculari. Infatti, questa loro caratteristica principale si riflette inoltre anche nel loro utilizzo. Infatti la regola d'oro prevede che **le chiamate ai loro metodi devono essere speculari tra la serializzazione e il processo inverso**.

I metodi di queste classi lanciano eccezioni checked, che il compilatore Java impone di gestire, altrimenti il codice non compila. Pertanto è obbligatorio gestire le eccezioni, utilizzando `try-catch-finally` o `try-with-resources`.

Le eccezioni checked coinvolte sono:

Operazione	Eccezione
Apertura stream	FileNotFoundException
Lettura / Scrittura	IOException
Deserializzazione	ClassNotFoundException

Nota Bene: dichiarare `throws` nel metodo (rimanda la gestione al chiamante), quindi non bisogna utilizzare `try-catch-finally` nel metodo che la implementa, bensì `try-with-resources`.


```
import java.io.*; // per brevità espositiva importo tutto
import java.util.Date;
```

```
public class WriteValues{
    public static void main (String [] args) {
        ObjectOutputStream oos = null;
        try{
            FileOutputStream out = new FileOutputStream("theTime");
            oos = new ObjectOutputStream(out);
            oos.writeObject("Today");
            oos.writeObject(new Date());
            oos.writeObject(4);
            oos.writeObject(3.14);
            oos.writeObject(true);
            oos.writeObject('k'); // Le catch vanno in questo ordine
        } catch (FileNotFoundException err) { err.getMessage();}
        catch (IOException err) { err.getMessage();}
        finally {
            if (oos != null) {
                try {oos.close();}
                catch (IOException err) {IO.println(err.getMessage());}
            }
        }
    }
}
```

```
import java.io.*;
import java.util.Date;
```

```
public class ReadValues{
    public static void main (String [] args) {
        ObjectInputStream ois = null;
        try {
            FileInputStream in = new FileInputStream("theTime");
            ois = new ObjectInputStream(in);
            String today = (String) ois.readObject();
            Date date = (Date) ois.readObject();
            int four = (Integer) ois.readObject();
            double pi = (Double) ois.readObject();
            boolean bool = (Boolean) ois.readObject();
            char ch = (Character) ois.readObject();
            IO.println(today + " " + date);
            IO.println(four + ", " + pi + ", " + bool + ", " + ch);
        } // Le catch vanno in questo ordine
        catch (FileNotFoundException err) { err.getMessage();}
        catch (IOException err) {IO.println(err.getMessage());}
        catch (ClassNotFoundException err) {IO.println(err.getMessage());}
        finally {
            if (ois != null) {
                try {ois.close();}
                catch (IOException err) {IO.println(err.getMessage());}
            }
        }
    }
}
```

L'esempio accanto mostra come si scrivono alcuni oggetti su di un ObjectOutputStream che incapsula il file (binario) theTime. L'oggetto new Date() rappresenta la data corrente (con precisione in millisecondi).

Se si scrive in un ObjectOutputStream con il metodo writeObject un oggetto che ha puntatori ad altri oggetti, allora tutti gli oggetti raggiungibili, sia direttamente che transitivamente, vengono scritti nello stream, in modo da mantenere le relazioni tra di essi.

Il metodo writeObject lancia un'eccezione NotSerializableException se cerca di serializzare un oggetto che non implementa l'interfaccia Serializable.

L'assenza di una estensione nel file di output è irrilevante, potevamo chiamarla "trallallero.trallallalla" e non avrebbe fatto alcuna differenza. JVM legge il binario del file, non l'estensione che gli affibbiamo.

Per leggere degli oggetti da uno stream, si usa in modo simmetrico la classe ObjectInputStream. Il prossimo esempio legge dal file chiamato theTime gli oggetti che erano stati scritti dal programma WriteValues.

Naturalmente gli oggetti devono essere riletti nell'ordine in cui erano stati scritti. **Nota bene:** Il metodo readObject può lanciare l'eccezione controllata ClassNotFoundException. Inoltre il tipo di readObject è Object e quindi serve un cast. E come è possibile notare, si usa il casting delle classi Wrapper e non dei tipi primitivi perché, prendiamo in esempio int, quando viene salvato da writeObject, viene scritto sotto forma di oggetto Integer. I metodi accennati più su, riferiti alla lettura di tipi primitivi, vanno in coppia.

La regola è semplice: **l'ordine e il tipo di ogni operazione di lettura deve specchiare esattamente l'ordine e il tipo di ogni operazione di scrittura.**

Le coppie compatibili sono:

Scrittura	Lettura
writeObject(obj)	readObject()
writeBoolean(boolean)	readBoolean()
writeByte(int)	readByte()
writeBytes(String)	readBytes()
writeShort(int)	readShort()
writeChar(int)	readChar()
writeChars(String)	readChars()
writeInt(int)	readInt()
writeLong(long)	readLong()
writeFloat(float)	readFloat()
writeDouble(double)	readDouble()
writeUTF(String)	readUTF()

Il metodo flush

Poiché stiamo scrivendo su file, esiste anche per la classe `ObjectOutputStream` un metodo che si assicura che il buffer di scrittura venga svuotato anche se non ancora pieno (condizione che ne innesca l'uso automatico).

Ci sono infatti delle volte per cui abbiamo bisogno di svuotare il buffer per essere sicuri di avere i dati effettivamente scritti, come nel caso di comunicazioni in tempo reale oppure per evitare delle attese lunghissime per l'effettiva scrittura da un buffer pieno o ancora per evitare di perdere dati sensibili per via di un eventuale crash o blackout dal sistema.

Si invoca così comè, senza argomenti, come metodo dell'oggetto `ObjectOutputStream` e lui pensa a tutto il resto. Il motivo per cui non si invoca solitamente è perché il metodo `close()` invoca automaticamente `flush()` prima della chiusura dello stream.

Attributi Transient e la variabile serialPersistentFields

Di norma, la serializzazione include tutti i campi di istanza di una classe, tranne quelli dichiarati come **static** o marcati con il modificatore **transient**.

Quest'ultimo serve a escludere esplicitamente un campo dal processo, ad esempio per motivi di sicurezza o perché non è serializzabile (come un oggetto `Thread`).

Tuttavia, Java offre anche un approccio alternativo e più restrittivo: il meccanismo basato sulla variabile **serialPersistentFields**, che per essere più chiari non deve portare per forza questo nome.

Infatti il segreto sta nella sua dichiarazione:

```
private static final ObjectOutputStream []
```

Se manca uno di questi modificatori, il campo viene ignorato, e si ricade nel comportamento predefinito.

//l'esempio precedente, se fosse scritto con i metodi primitivi:

// SCRITTURA con metodi primitivi

```
oos.writeUTF("Today"); // solo con i String
oos.writeObject(new Date()); // con qualunque oggetto
oos.writeInt(4); // solo con int/Integer
oos.writeDouble(3.14); // solo con double/Double
oos.writeBoolean(true); // solo con boolean/Boolean
oos.writeChar('k'); // solo con char/Char
```

// LETTURA — deve usare le stesse coppie

```
String today = ois.readUTF(); // simmetrico a writeUTF
Date date = (Date) ois.readObject(); // simmetrico a writeObject
int four = ois.readInt(); // simmetrico a writeInt
double pi = ois.readDouble(); // simmetrico a writeDouble
boolean bool = ois.readBoolean(); // simmetrico a writeBoolean
char ch = ois.readChar(); // simmetrico a writeChar
```

come è possibile notare, usare i `write` e `read` specifici del tipo primitivo non obbliga al casting ed possibile fargli accogliere anche le rispettive classi Wrapper.

Da notare poi `writeBytes` e `writeChars` che non ricevono un `int`, ma uno `String`.

```
public class Persona implements java.io.Serializable {
    private String nome, cognome;
    private transient Thread t = new Thread();
    private transient String codiceSegreto;
    private static final ObjectOutputStream[] serialPersistentFields
        = {new ObjectOutputStream("nome", String.class),
            new ObjectOutputStream("codiceSegreto", String.class)};
    public Persona(String nome, String cognome, String cs) {
        this.setNome(nome);
        this.setCognome(cognome);
        this.setCodiceSegreto(cs);
    }
    // Metodi setter e getter omissi
    public String toString() {
        return "Nome: " + getNome() + "\nCognome: " +
            getCognome() + "\nCodice Segreto: " +
            getCodiceSegreto();
    }
}
```


Questo sistema consente di dichiarare in modo esplicito quali campi devono essere serializzati, ignorando tutti gli altri. È una strategia di tipo white list, che sovrascrive completamente la logica predefinita di esclusione (black list) basata su transient.

Nel codice sopra, solo i campi nome e codiceSegreto vengono inclusi nella serializzazione, mentre cognome (pur non essendo dichiarato transient) viene ignorato. Questo perché la variabile serialPersistentFields sovrascrive completamente la logica predefinita del meccanismo basato su transient. In altre parole, quando si usa la white list, **solo i campi esplicitamente indicati vengono serializzati, tutti gli altri vengono esclusi**.

Possiamo verificare facilmente il comportamento del meccanismo con un semplice esempio di serializzazione e deserializzazione con le seguenti classi:

```
import java.io.*;
public class SerializeObject {
    public static void main(String[] args) throws IOException {
        Persona p = new Persona("Mattia", "Pascal", "A131eBrazow");
        try (FileOutputStream f = new FileOutputStream("persona.ser");
            ObjectOutputStream s = new ObjectOutputStream(f)) {
            s.writeObject(p);
            IO.println("Oggetto serializzato!");
        }
    }
}
```

Eseguendo il programma di deserializzazione otteniamo il seguente output:

```
Oggetto deserializzato!
Nome: Mattia
Cognome: null
Codice Segreto: A131eBrazow
```

Possiamo notare che il campo cognome risulta null: non è stato incluso nel flusso di byte, perché non fa parte della white list definita da serialPersistentFields.

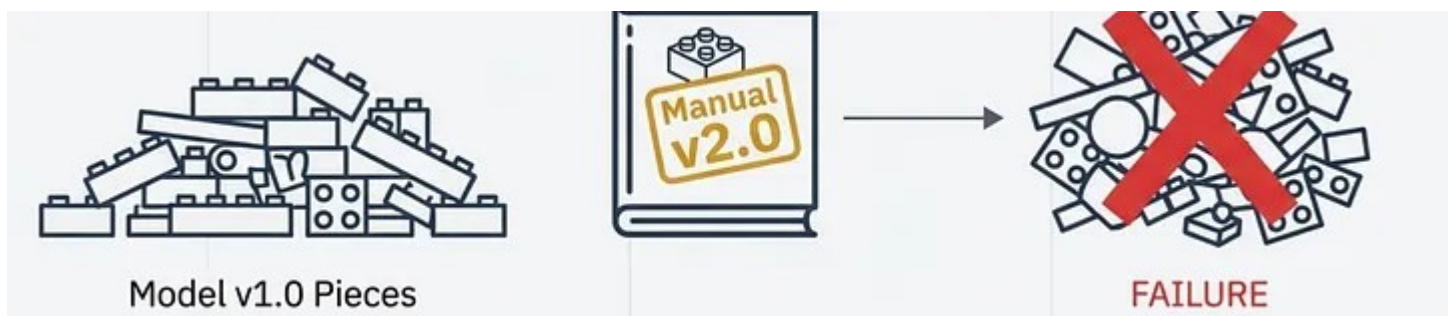
```
import java.io.*;
public class DeSerializeObject {
    public static void main(String[] args) throws Exception {
        Persona p = null;
        try (FileInputStream f = new FileInputStream("persona.ser");
            ObjectInputStream s = new ObjectInputStream(f)) {
            p = (Persona) s.readObject();
            IO.println("Oggetto deserializzato!");
            IO.println(p);
        }
    }
}
```

Questo approccio può risultare utile in contesti in cui è necessario un controllo rigoroso o centralizzato dei dati che devono essere salvati o trasmessi, come in applicazioni sensibili o versionabili. Tuttavia, nella maggior parte dei casi, l'uso di transient è più chiaro e leggibile, poiché mostra direttamente nel codice quali campi non devono essere serializzati.

Compatibilità tra le versioni: la serialVersionUID

La serialVersionUID è un identificatore univoco che garantisce che stiamo cercando di deserializzare il file con la stessa versione (o con una compatibile) della classe che descriveva l'oggetto in questione.

Tornando all'esempio del modellino LEGO. Possiamo immaginarlo come se, pur avendo gli stessi pezzi a disposizione, avessimo la seconda versione del manuale, ma questo non ci permetterebbe di ricostruire il modellino perché posso tirare fuori dalla scatola un pezzo alla volta per poi cercare di incastrarlo in punti in cui non può entrare, ottenendo un vero disastro.



Inizializzazione della serialVersionUID

La dichiarazione e la inizializzazione sono in realtà piuttosto semplici; basta aggiungere questa riga di codice:

```
private static final long serialVersionUID = 1L;
```

dove 1 sta ad indicare la versione della classe di riferimento, in questo caso intende la prima versione.

Se fosse una seconda versione ci sarebbe scritto 2L.

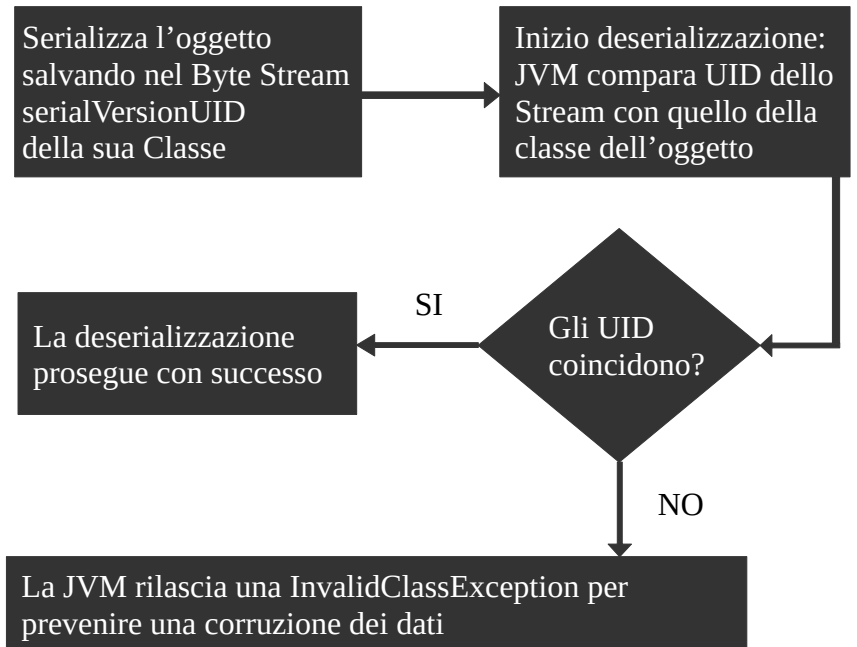
Se non la inserisci, la JVM ci penserà al posto tuo, a tuo rischio e pericolo.

Come JVM utilizza la serialVersionUID

- Durante la serializzazione il serialVersionUID è salvato come parte dello Stream;
- Quando deve eseguire della deserializzazione la JVM esegue un semplice controllo all'inizio, comparando il serialVersionUID della classe correntemente caricata nell'applicazione.
- Se il controllo è positivo, allora esegue la deserializzazione, altrimenti rilascia un `java.io.InvalidClassException`.

Nota: l'errore nei log sarà simile a questo:

```
java.io.InvalidClassException: MyClass; local class incompatible: stream classdesc serialVersionUID = 1, local class serialVersionUID = 2
```



Due possibili scelte: generare il proprio UID o fare in modo che la JVM lo generi per noi

Esiste un modo professionale e affidabile per gestire questa situazione, ma c'è anche il comportamento fragile e predefinito che spesso causa problemi di produzione.



È piuttosto chiaro che sia preferibile consigliabile obbligatorio (in ambito professionale) dichiarare la propria variabile UID per tre ragioni principali:

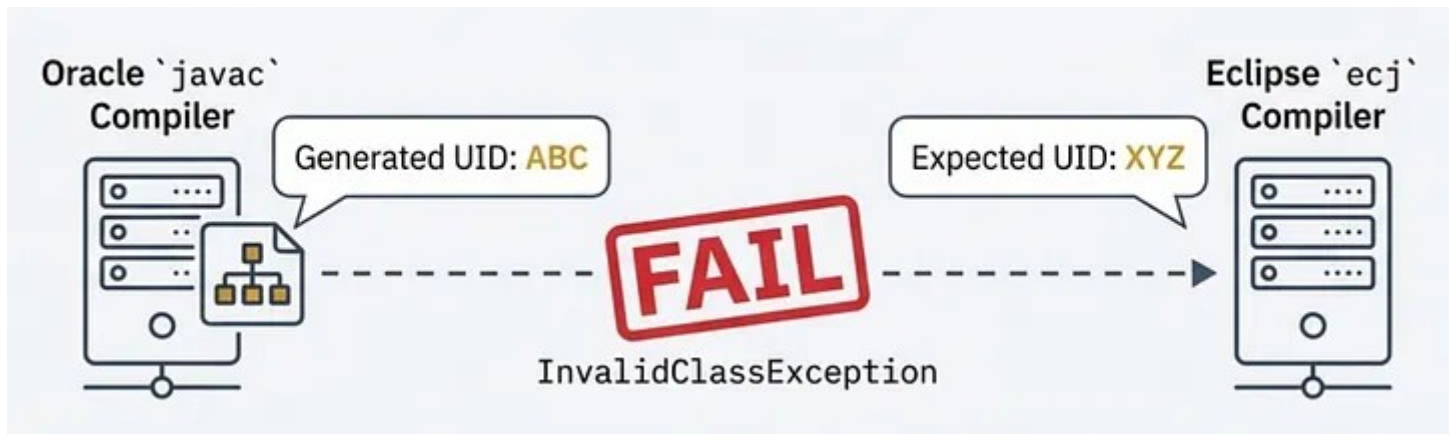
- **Controllo delle versioni:** dichiarando l'UID, sei tu a decidere. Puoi apportare modifiche "compatibili", come l'aggiunta di un nuovo campo, senza compromettere il sistema. Quando deserializzi un vecchio oggetto, la JVM inizierà semplicemente il nuovo campo al suo valore predefinito. L'UID si modifica solo quando si apporta una modifica incompatibile, come la modifica di un campo da int a string.

```
// versione 1.0
class User Implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
}
```

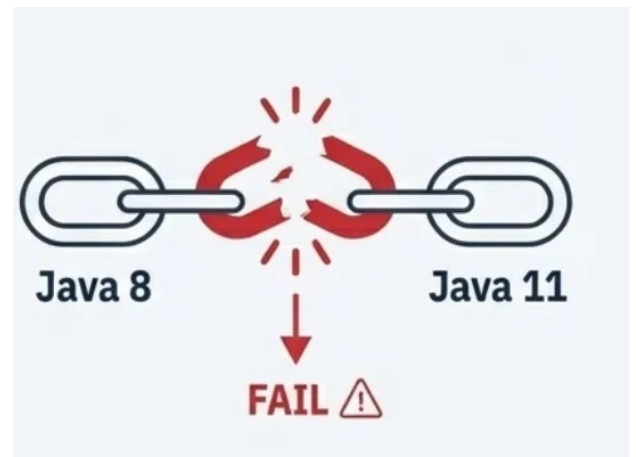
Cambiamento compatibile
(aggiunto campo 'email')

```
// Versione 1.1
class User Implements
Serializable {
    private static final
long serialVersionUID =
1L; // invariato
    String name;
    String email; // Nuovo
campo
    ...
}
```

- **Coerenza tra gli ambienti:** nei sistemi distribuiti, macchine diverse potrebbero utilizzare compilatori diversi (ad esempio, javac di Oracle rispetto a ecj di Eclipse). Poiché questi compilatori generano bytecode in modo diverso, potrebbero calcolare UID "automatici" diversi per lo stesso identico codice sorgente. La dichiarazione dell'UID risolve completamente questo problema.

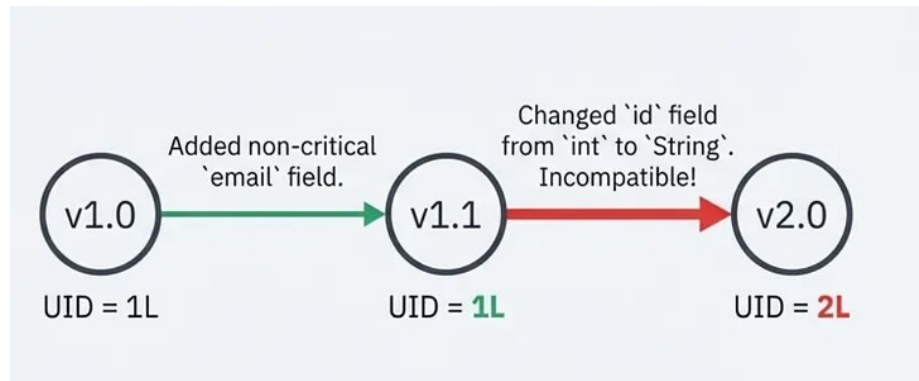


- **Prevenire sorprese future:** l'aggiornamento delle versioni di Java può compromettere gli UID non dichiarati. Ad esempio, il passaggio da Java 8 a Java 11 ha modificato il modo in cui venivano generati i "metodi di accesso sintetici". Poiché questi metodi nascosti fanno parte del calcolo automatico degli UID, molte aziende hanno dovuto affrontare interruzioni di produzione perché i loro UID sono cambiati semplicemente perché avevano aggiornato il compilatore.



Inizia con 1L e sii consapevole

Il valore 1L è un punto di partenza perfetto. Indica che ti stai assumendo la responsabilità del versioning. Cambialo in 2L solo se vuoi intenzionalmente interrompere la compatibilità con tutti i dati salvati in precedenza.



L'interfaccia Externalizable (un'estensione di Serializable)

Dalla domanda "Posso avere ancora più controllo sui dati che voglio serializzare?" nasce **java.io.Externalizable**. Questa interfaccia si configura come uno strumento per gestire in modo ancora più accurato la serializzazione. Anche questa è una interfaccia Marker (senza metodi) e serve solo a dire alla JVM che la classe avrà all'interno due metodi in override: **writeExternal(ObjectOutputStream output)** e **readExternal(ObjectInputStream input)**. A differenza di **Serializable**, dove la JVM gestisce automaticamente i campi non transient, con **Externalizable** è il programmatore a decidere cosa scrivere e cosa leggere, campo per campo.

- **Non ha bisogno di attributi Transient o serialPersistentFields** perché è il programmatore a decidere (all'interno dei due metodi sopra citati) cosa serializzare.
- **Gestisce la serialVersionUID**, che è l'unica cosa che viene salvata (e generata se assente) automaticamente.
- Durante la deserializzazione **ha bisogno** della presenza **di un costruttore senza argomenti** all'interno della classe da poter essere chiamato prima di **readExternal()**, se non esiste il programma lancia una **InvalidClassException**.
- **I metodi di lettura e scrittura sono gli stessi utilizzati con Serializable**, proprio perché gli oggetti che vengono utilizzati sono gli stessi (**ObjectInputStream** e **ObjectOutputStream**).
- Come per **Serializable**, per ereditarietà **bisogna implementare Externalizable anche in tutte le sottoclassi**.

Quando usare Externalizable

È preferibile a **Serializable** quando si ha bisogno di performance elevate (scrivere solo i campi strettamente necessari), quando si vuole cifrare o trasformare i dati prima di scriverli, o quando la classe ha una struttura complessa che la serializzazione automatica non gestisce bene.

Vulnerabilità derivanti dalla Serializzazione Binaria

A detta di personalità quali Mark Reinhold e Brian Goetz, la serializzazione in Java presenta gravi problemi di sicurezza, soprattutto durante la deserializzazione di dati non attendibili.

- **Esecuzione di codice remoto (RCE)**: Un attaccante può costruire un payload malevole che, una volta deserializzato, esegue codice arbitrario. Questo avviene sfruttando catene di metodi chiamate **deserialization gadgets**.
- **Denial of Service (DoS)**: oggetti progettati per consumare grandi quantità di memoria o causare cicli infiniti possono bloccare l'applicazione.
- **Fuga di dati sensibili**: campi privati possono essere esposti se non gestiti correttamente, anche se dichiarati transient.
- **Iniezione di oggetti**: è possibile inserire oggetti non previsti nel flusso, alterando il comportamento dell'applicazione.

Oracle stessa ha definito la serializzazione un "errore" e ne sta rimuovendo il supporto perché responsabile di circa un terzo delle vulnerabilità di sicurezza nella piattaforma Java (Project Amber). Per mitigare i rischi:

- Evitare la deserializzazione di dati provenienti da fonti non fidate.
- Usare il filtro di deserializzazione (disponibile da Java 9) per limitare le classi deserializzabili, come ad esempio `System.setProperty("jdk.serialFilter", "allowedClass;!*");`
- Preferire formati alternativi come JSON, Protocol Buffers o XML per comunicazioni esterne.
- Criptare i dati serializzati prima del salvataggio.
- Usare **ValidatingObjectInputStream** da Apache Commons IO per controllare i tipi.

La Serializzazione Testuale

In Java la serializzazione testuale (al posto del suo formato binario nativo di Java) si riferisce alla conversione di oggetti Java in un formato testuale leggibile, come JSON (JavaScript Object Notation) o XML (Extensible Markup Language), usati per gli stessi scopi, ma che differiscono in sintassi, uso e prestazioni.

Questo approccio è utile per lo scambio di dati tra sistemi, API REST, configurazioni e log.

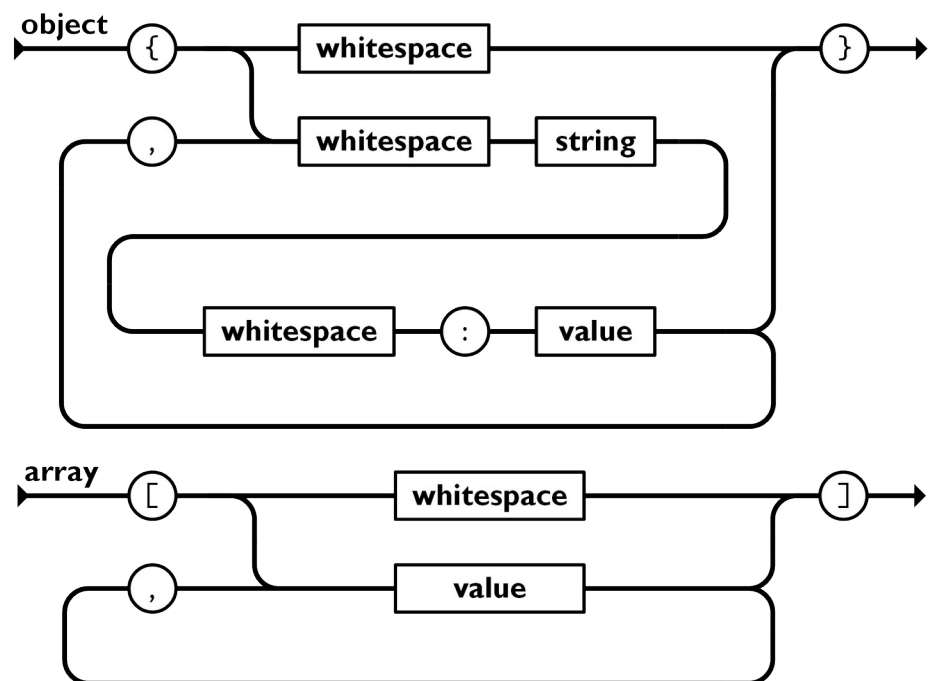
Il formato JSON

JSON (JavaScript Object Notation) è un semplice formato per lo scambio di dati. Per le persone è facile da leggere e scrivere, mentre per le macchine risulta facile da generare e analizzarne la sintassi. Si basa su un sottoinsieme del Linguaggio di Programmazione JavaScript, Standard ECMA-262 Terza Edizione - Dicembre 1999. Tra gli Standard che lo definiscono ci sono ISO/IEC 21778:2017 e ECMA-404 (2013), che ne descrivono solo la sintassi, mentre la RFC (Request for Comments) copre alcune considerazioni sulla sicurezza e l'interoperabilità.

È un formato di testo completamente indipendente dal linguaggio di programmazione, ma utilizza convenzioni conosciute dai programmatori di linguaggi della famiglia del C, come C, C++, C#, Java, JavaScript, Perl, Python, e molti altri. Questa caratteristica fa di JSON un linguaggio ideale per lo scambio di dati.

JSON è basato su due strutture, **che possono essere annidate**:

- **Un insieme non ordinato di coppie nome/valore.** In diversi linguaggi, questo è realizzato come un **oggetto**, un record, uno struct, un dizionario, una tabella hash, un elenco di chiavi o un array associativo. Inizia con {parentesi graffa sinistra e finisce con }parentesi graffa destra. Ogni nome è seguito da : (due punti) e le coppie nome/valore sono separate da , (virgola).
- **Un elenco ordinato di valori.** Nella maggior parte dei linguaggi questo si realizza con un array, un vettore, un elenco o una sequenza. Comincia con [(parentesi quadra sinistra) e finisce con] (parentesi quadra destra). I valori sono separati da , (virgola).



I tipi di dati (valori) di base di JSON sono:

Stringa: è una raccolta di zero o più caratteri Unicode, tra virgolette doppie; per le sequenze di escape utilizza la barra rovesciata. Un singolo carattere è rappresentato come una stringa di caratteri di lunghezza uno. Una stringa è molto simile ad una stringa C o Java.

Numero: è molto simile ad un numero C o Java, a parte il fatto che i formati ottali e esadecimali non sono utilizzati. Un numero decimale con segno che può contenere una parte frazionaria e può utilizzare la notazione esponenziale E, ma non può includere valori non numerici come NaN.

Booleano: uno dei valori true o false.

Array: un elenco ordinato di zero o più elementi, ognuno dei quali può essere di qualsiasi tipo.

Oggetto: una raccolta di coppie nome-valore in cui i nomi (chiamati anche chiavi) sono stringhe.

null: un valore vuoto, che utilizza la parola null

Spazi vuoti: ignorati (non nelle stringhe). Sono spazio, tabulazione orizzontale, avanzamento riga e ritorno a capo.

Il seguente esempio mostra una possibile rappresentazione JSON della descrizione di una persona

Poiché non è affatto scontato che la sequenza JSON generata sia formattata con tabulazioni e ritorni a capo, la cosa che può tranquillamente accadere è di ritrovarsi con un file che sostanzialmente è una lunga stringa (su un'unica riga). Per venire incontro alla leggibilità umana, esistono strumenti on-line che permettono di verificare la correttezza del JSON che abbiamo sotto mano. Uno dei tanti è jsonformatter.curiousconcept.com

Uno dei modi più comuni in Java per effettuare la serializzazione testuale in formato JSON è utilizzare librerie esterne come Jackson o Gson.

La libreria Jackson

Gestita dal progetto maven, è costituita sostanzialmente da tre pacchetti:

- ➔ jackson-core
- ➔ jackson-annotations
- ➔ jackson-databind

Ci sono due (e mezzo) possibili strade per acquisire questa libreria:

- <https://central.sonatype.com/> : sito ufficiale
- <https://github.com/FasterXML> : sito dello sviluppatore
- <https://mvnrepository.com/repos/central> : sito non ufficiale (utile per .jar)

1) La prima strada si compone di molte buone intenzioni, cioè **gestire il progetto maven da zero con all'interno le dipendenze direttamente inserite nel file pom.xml**, il quale dirà automaticamente alla JDK cosa deve fare durante la compilazione del progetto, per questo passaggio avremo bisogno della sola jackson-databind perché porta con sé le dipendenze dirette delle altre due, quindi ci penserà la JDK a completarle:

Quindi bisogna anzitutto andare a ricercare, tramite i link ufficiali, jackson-databind, dove verrà fornito, nella sezione snippets, il seguente codice ➔ che dovrà essere copiato/incollato all'interno del file pom.xml (in NetBeans → opzioni sul nome → Open Pom) del nostro progetto all'interno dei marker

<dependencies> qui dentro </dependencies>, ottenendo un risultato del tipo:

```
{
  "name" : "Ajeje",
  "last_name" : "Brazow",
  "is_alive" : true,
  "age" : 27,
  "address" : {
    "street" : "Enrico Fermi",
    "number" : 28,
    "city" : "Caltanissetta",
    "state" : "Italia",
    "postal_code" : 93100
  },
  "phone_numbers" : [
    {
      "type" : "home",
      "number" : "093474111"
    },
    {
      "type" : "office",
      "number" : null
    }
  ],
  "children" : [
    "Michele"
  ],
  "spouse" : null
}
```

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>numero_versione</version>
</dependency>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>md</groupId>
  <artifactId>ProgettoEsempio</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.21.1</version> ← in questo caso è stata inserita la versione 2.21.1
    </dependency>
  </dependencies>
</project>
```


Fatto ciò, bisogna effettuare (tasto opzioni su nome del progetto) un **Clean** e successivamente un **Build with Dependencies**. A questo punto potremo notare la presenza di una cartella Dependencies all'interno del progetto. Da qui in poi sarà possibile utilizzare la libreria jackson per i nostri scopi.

Nota bene: questo metodo richiede l'utilizzo di una connessione internet almeno per la prima compilazione, per fare in modo che possano essere inserite nella tua cache locale Maven (~/.m2/repository). Quindi il problema si presenta solo se la cache non ha questo repository in locale, perché JDK cercherà di contattare il server e, se non c'è una connessione presente, darà problema.

Nota bene: all'interno di NetBeans questo metodo è indispensabile se il nostro progetto è già Maven.

1.5) NetBeans mette a disposizione la possibilità di reperire le librerie Maven direttamente dall'IDE. Creato il progetto Maven, non serve altro che andare sulla cartella Dependencies → tast. opzioni → add Dependency → nella casella query inserire "jackson" e verranno filtrati i pacchetti cercati. Come il precedente, basta selezionare databind.

2) Il secondo metodo (che **funziona anche con i progetti NON-Maven**) si rifà a un semplice **copia/incolla dei file .jar** all'interno di una cartella lib all'interno del progetto cui stiamo lavorando.

Per acquisire i tre file possiamo avvalerci di due metodi:

- il primo metodo è direttamente collegato a quelli sopra esposti perché durante la build ha creato i tre pacchetti .jar che ci servono. Infatti, all'interno della cache .m2 (file nascosto), avvalendoci del percorso
~/.m2/repository/com/fasterxml/jackson/core/ la ~ indica la home in Unix e NomeUtente in Windows

troveremo le tre cartelle che contengono ognuna i pacchetti .jar necessari.

- il secondo metodo si rifà al semplice scaricamento dei tre file .jar dal sito non ufficiale sopracitato.

Acquisiti pertanto i file, possiamo incollarli all'interno di una cartella lib (crearla se non c'è) all'interno della cartella principale del nostro progetto.

Con Eclipse → tast. opzioni sul nome del progetto → build path → add external archives → spostarsi all'interno della cartella lib del progetto → selezionare i tre .jar (tast. ctrl per multiselezione) → add

Con NetBeans → aprire l'albero del progetto → tast. opzioni sulla cartella Libraries → add jar/folder → spostarsi all'interno della cartella lib del progetto → selezionare i tre .jar (tast. ctrl per multiselezione) → add

Nota bene: quando si apre il percorso, è plausibile che rimandi a una cartella lib di un altro progetto già aperto in precedenza; se qui sono presenti i tre .jar richiesti, non fa alcuna differenza prendere quelli piuttosto che gli altri che avevamo copiato nella cartella principale del progetto.

Ricordo che questo metodo su NetBeans non funziona se il progetto è già Maven.

Fatto ciò sarà possibile finalmente utilizzare la libreria Jackson (per verificare ciò, provare a scrivere il comando specifico `ObjectMapper mapper = new ObjectMapper ();` che non dovrebbe dare problemi).

Nota Bene: ogni volta che si esegue l'importazione di una libreria .jar, questa viene comunque caricata all'interno della cache locale Maven, pertanto, anche volendo, non c'è bisogno assoluto di spostare i file .jar all'interno di una cartella lib del progetto. Questo metodo è fatto solo per rendere semplice il percorso.

Nota Molto Bene: l'importazione di ogni libreria Maven (quindi non soltanto jackson) avviene in questo modo.

ObjectMapper

È la classe principale di Jackson, il punto di ingresso per tutte le operazioni di serializzazione (Java → JSON) e deserializzazione (JSON → Java).

È una classe thread-safe dopo la configurazione, quindi la cosa migliore da fare è crearla una volta sola con

```
ObjectMapper mapper = new ObjectMapper();
```

Metodi principali

Serializzazione (Java → JSON):

writeValue è un metodo in overload

```
// Oggetto → stringa di JSON
String json = mapper.writeValueAsString(oggetto);
// Oggetto → file JSON
mapper.writeValue(new File("output.json"), oggetto);
// Oggetto → byte array
byte [] bytes = mapper.writeValueAsBytes(oggetto);
// Serializzazione "pretty printed" (indentata)
String writerWithDefaultPrettyPrinter()
    .writeValueAsString(Object value)
```

Deserializzazione (JSON → Java):

come si può notare,

readValue è in overload

```
// Stringa JSON → oggetto
MyClass obj = mapper.readValue(jsonString, MyClass.class);
// File JSON → oggetto
MyClass obj = mapper.readValue(new File("input.json"), MyClass.class);
// JSON → lista
List<MyClass> lista = mapper.readValue(jsonString,
    new TypeReference<List<MyClass>>() {});
// JSON → mappa
Map<String, Object> mappa = mapper.readValue(jsonString,
    new TypeReference<Map<String, Object>>() {});
```

Conversione diretta (Java → Java):

convertValue è in overload

```
// Convertire un oggetto in un altro tipo
MyClass obj = mapper.convertValue(altroOggetto, MyClass)
```

Lavorare con JsonNode (albero JSON):

readTree è in overload

```
// Legge JSON come albero navigabile
JsonNode node = mapper.readTree(jsonString);
String valore = node.get("campo").asText();
```

Configurazione comune:

```
mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
mapper.configure(SerializationFeature.INDENT_OUTPUT, true);
mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);
```

Gestione eccezioni

ObjectMapper può lanciare principalmente queste eccezioni, tutte figlie di JacksonException:

Eccezione	Quando viene lanciata
<code>JsonParseException</code>	JSON malformato
<code>JsonMappingException</code>	impossibile mappare JSON sulla classe Java
<code>JsonProcessingException</code>	eccezione generica padre delle precedenti

Gestione try-catch-finally

Caso base con stringa:

```
ObjectMapper mapper = new ObjectMapper();
try {
    MyClass obj = mapper.readValue(jsonString, MyClass.class);
    System.out.println(obj);
} catch (JsonParseException e) {
    // JSON sintatticamente non valido
    System.err.println("JSON malformato: " + e.getMessage());
} catch (JsonMappingException e) {
    // Campi non compatibili con la classe
    System.err.println("Errore di mapping: " + e.getMessage());
} catch (JsonProcessingException e) {
    // Qualsiasi altro errore di processing
    System.err.println("Errore generico: " + e.getMessage());
}
```

Caso con file, dove il finally è importante:

```
ObjectMapper mapper = new ObjectMapper();
FileInputStream fis = null;
try {
    fis = new FileInputStream("input.json");
    MyClass obj = mapper.readValue(fis, MyClass.class);
    System.out.println(obj);
} catch (JsonParseException e) {
    System.err.println("JSON malformato: " + e.getMessage());
} catch (JsonMappingException e) {
    System.err.println("Errore di mapping: " + e.getMessage());
} catch (IOException e) {
    System.err.println("Errore I/O: " + e.getMessage());
} finally {
    // Il finally garantisce che lo stream venga sempre chiuso
    if (fis != null) {
        try {
            fis.close();
        } catch (IOException e) {
            System.err.println("Errore chiusura stream: " + e.getMessage());
        }
    }
}
```

Versione moderna con try-with-resources (consigliata):

```
ObjectMapper mapper = new ObjectMapper();
try (FileInputStream fis = new FileInputStream("input.json")) {
    MyClass obj = mapper.readValue(fis, MyClass.class);
    System.out.println(obj);
} catch (JsonParseException e) {
    System.err.println("JSON malformato: " + e.getMessage());
} catch (JsonMappingException e) {
    System.err.println("Errore di mapping: " + e.getMessage());
} catch (IOException e) {
    System.err.println("Errore I/O: " + e.getMessage());
}
// lo stream viene chiuso automaticamente, il finally non serve
```

Un breve esempio esplicativo

```
public class Persona {
    private String nome;
    private int eta;

    public Persona() {} // obbligatorio!

    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
    // getter e setter obbligatori per ogni attributo
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public int getEta() { return eta; }
    public void setEta(int eta) { this.eta = eta; }
}
```

```
public class Ordine {
    private int id;
    // oggetto annidato
    private Persona cliente;
    // lista di oggetti
    private ArrayList<Prodotto> prodotti;

    public Ordine() {}

    public Ordine(int id, Persona cliente) {
        this.id = id;
        this.cliente = cliente;
        // ArrayList inizializzato internamente
        this.prodotti = new ArrayList<Prodotto>();
    }

    // getter e setter per tutti i campi...
}
```

La libreria jackson si gestisce tramite i **costruttori vuoti** e i **getter** e **setter**. Questa regola vale **per ogni classe coinvolta** nella trascrizione. ObjectMapper serializza automaticamente i campi pubblici o quelli accessibili tramite getter. Non usa toString() per la serializzazione JSON, a meno che non lo si configuri esplicitamente.

```
// try-catch-finally volutamente omessi in questo esempio
// per semplicità espositiva la classe Prodotti la si dà come copia di Persona
ObjectMapper mapper = new ObjectMapper();

Persona pers1 = new Persona("Mario", 30);

//Oggetto → JSON (scrittura)
// Come stringa
String json = mapper.writeValueAsString(pers1);
IO.println(json); // {"nome":"Mario","eta":30}

// Direttamente su file
mapper.writeValue(new File("persona.json"), pers1);
// senza un percorso assoluto genera il file nella cartella radice del progetto

//JSON → Oggetto (lettura)
// Da stringa
String json = "{\"nome\":\"Mario\",\"eta\":30}";
Persona pers2 = mapper.readValue(json, Persona.class);

// Da file
Persona pers3 = mapper.readValue(new File("persona.json"), Persona.class);

IO.println(pers2.getNome()); // "Mario"

//ArrayList → JSON e viceversa
// Lista → JSON
ArrayList<Persona> persone = new ArrayList<>();
persone.add(new Persona("Mario", 30));
persone.add(new Persona("Luigi", 25));

String json = mapper.writeValueAsString(persone);
// [{"nome":"Mario","eta":30},{"nome":"Luigi","eta":25}]

// JSON → Lista
List<Persona> lista = mapper.readValue(
    json,
    new TypeReference<List<Persona>>() {}
);
// oggetti che contengono oggetti
Ordine ord = new Ordine(42, pers1);
ord.getProdotti().add(new Prodotto("Penna", 1.50));
ord.getProdotti().add(new Prodotto("Quaderno", 3.00));
//per non ripeterci e per far vedere altri comandi, usiamo il metodo che indenta
mapper.writerWithDefaultPrettyPrinter().writeValue(new File("ordine.json"), ord);
```

```
{
  "id" : 42,
  "cliente" : {
    "nome" : "Mario",
    "eta" : 30
  },
  "prodotti" : [
    {
      "nome" : "Penna",
      "prezzo" : 1.50
    },
    {
      "nome" : "Quaderno",
      "prezzo" : 3.00
    }
  ]
}
```



Quando toString() è rilevante:

Alle volte c'è bisogno di effettuare un override del metodo toString() per utilizzare al meglio la libreria per le proprie esigenze specifiche. In quei casi c'è bisogno di utilizzare alcune piccole accortezze.

1. Con @JsonValue: Se annoti toString() con @JsonValue, Jackson lo userà per serializzare l'oggetto.

```
@Override
@JsonValue
public String toString() {
    return nome + " " + cognome;
}
```

2. Per enum: Puoi usare WRITE_ENUMS_USING_TO_STRING per far sì che Jackson usi toString() invece del nome costante.
3. Serializzazione personalizzata: Se crei un JsonSerializer che chiama toString().

Ma per questa panoramica generale non mi addenterò troppo nella questione perché richiederebbe l'uso di svariate righe di codice come esempio. A tal proposito, nelle fonti è presente una lista di articoli dello stesso autore che entra più nel dettaglio nell'uso di questa libreria, con esempi molto mirati e chiari.

Inoltre, bisogna tenere sempre a mente che questa è una libreria, che per quanto sia largamente utilizzata in ambito professionale, non rappresenta l'unica libreria disponibile. Google ha infatti sviluppato il proprio traduttore, Gson, il quale ha i propri comandi e le proprie logiche. Si rimanda sempre alle guide ufficiali e ai forum per comprenderne l'uso in caso di necessità.

Il formato XML

XML (eXtensible Markup Language) è un formato molto simile a HTML per struttura e sintassi, ma, a differenza di quest'ultimo, è molto rigido sulla sintassi da seguire. Utilizza dei marcatori, chiamati tag (etichette), per assegnare una semantica al testo. I tag possono contenere informazioni in due modi: attraverso dei parametri oppure racchiudendo del testo o altri tipi di informazioni. Segue che possono essere tag di apertura, necessariamente seguiti da tag di chiusura (tra i quali si può avere un contenuto) oppure tag che si aprono e chiudono, e possono quindi fornire informazioni solo attraverso i loro parametri. Ogni etichetta inizia e finisce con delle parentesi uncinate (<>) (che in altri contesti sarebbero i segni di minore e maggiore), mentre la chiusura del tag o il tag di chiusura è rappresentato dalla barra (/).

Anche qui le librerie utilizzare per la traduzione sono varie. Tra le più utilizzate ci sono:

- ➔ **JAXB**: la più usata per oggetti → XML.
È inclusa in Java (fino a Java 8), per Java 9+ va aggiunta come dipendenza.
- ➔ **Jackson XML**: se conosci già Jackson
Stessa sintassi di Jackson JSON, cambia solo la dipendenza!

Non mi addentro ulteriormente perché sarebbe una ripetizione di quanto detto per la libreria jackson json.

XML vs JSON — differenze rapide

Utilizzare JSON quando:

- ◆ **Creazione di API REST** – Standard di settore per le API web
- ◆ **Applicazioni web** – Supporto JavaScript nativo
- ◆ **App mobili** – Payload più piccolo = minore larghezza di banda
- ◆ **File di configurazione** – package.json, tsconfig.json, ecc.
- ◆ **Database NoSQL** – MongoDB, CouchDB utilizzano formati simili a JSON
- ◆ **Sistemi in tempo reale** – Analisi più rapida critica
- ◆ **Strutture dati semplici** – Oggetti e array

Utilizzare **XML** quando:

Markup di documenti – HTML, SVG, documenti Office

Servizi web SOAP – Integrazione aziendale

Validazione complessa – Requisiti rigorosi dello schema XSD

Contenuto misto – Testo con markup incorporato

Dati ricchi di metadati – Attributi per i metadati

Sistemi legacy – Infrastruttura XML esistente

Esigenze di trasformazione – XSLT per la trasformazione dei dati

Requisiti dello spazio dei nomi – Più vocabolari in un unico documento

Fonti utilizzate

Persistenza degli oggetti

[https://it.wikipedia.org/wiki/Persistenza_\(informatica\)](https://it.wikipedia.org/wiki/Persistenza_(informatica))

Serializzazione e deserializzazione in informatica

<https://it.wikipedia.org/wiki/Serializzazione>

<https://it.wikipedia.org/wiki/Deserializzazione>

https://it.wikipedia.org/wiki/Chiamata_di_procedura_remota

https://it.wikipedia.org/wiki/Common_Object_Request_Broker_Architecture

<https://learn.microsoft.com/it-it/dotnet/visual-basic/programming-guide/concepts/serialization/>

Serializzazione e deserializzazione Binaria in Java

<https://docs.oracle.com/javase/7/docs/platform/serialization/spec/serialTOC.html>

<https://docs.oracle.com/javase/8/docs/platform/serialization/spec/class.html>

<https://codescoddler.medium.com/unlocking-java-serialization-a-beginners-guide-to-serialversionuid-ee6b307dd417>

<https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/io/ObjectOutputStream.html>

<https://www.tecninfo.altervista.org/j3/index.php/dispense-oop-e-java/16-persistenza?start=5>

<https://codegym.cc/it/groups/posts/it.240.qual-e-la-differenza-tra-serializzazione-e-deserializzazione-in-java->

<https://programmingacademy.it/2020/05/serializzazione-e-deserializzazione-in-java/>

<https://www.ibm.com/docs/it/wxs/8.6.1?topic=overview-serialization>

<https://pages.di.unipi.it/corradini/Didattica/LIP-07/JavaIO/Serialization/main.html>

https://digilander.libero.it/CORSOLII/L24_Serialization.pdf

ObjectOutputStream e ObjectOutputStream

https://www.tutorialspoint.com/java/io/datainputstream_readutf.htm

<https://www.codejava.net/java-se/file-io/java-io-fileinputstream-and-fileoutputstream-examples>

<https://codegym.cc/it/groups/posts/it.245.inputoutput-in-java-classi-fileinputstream-fileoutputstream-e-bufferedinputstream>

<https://www.jmdoudoux.fr/java/dej/chap-flux.htm>

Approfondimenti: serialPersistentFields, serialVersionUID e interfaccia Externalizable

<https://www.claudiodesio.com/ser-wl/>

<https://www.geeksforgeeks.org/java/serialversionuid-in-java/>

<https://deepsources.com/directory/java/issues/JAVA-E1042>

<https://senoritadeveloper.medium.com/serialization-and-deserialization-in-java-what-is-serialversionuid-and-when-to-regenerate-it-22e1ee628675>
<https://intellipaat.com/blog/what-is-a-serialversionuid/>
<https://javarevisited.blogspot.com/2012/01/serializable-externalizable-in-java.html>
<https://www.tutorialspoint.com/article/difference-between-serialization-and-externalization-in-java>
<https://www.baeldung.com/java-externalizable>

Discussioni sul tema

[https://www.reddit.com/r/javahelp/comments/1cdo4k2/explain like im five what is serializable/?tl=it](https://www.reddit.com/r/javahelp/comments/1cdo4k2/explain-like-im-five-what-is-serializable/?tl=it)
[https://www.reddit.com/r/java/comments/1k4xz6c/how to deal with nonserializable fields in java/?tl=it](https://www.reddit.com/r/java/comments/1k4xz6c/how-to-deal-with-nonserializable-fields-in-java/?tl=it)
[https://www.reddit.com/r/learnjava/comments/18b0oal/questions about jsongson serialization and/?tl=it](https://www.reddit.com/r/learnjava/comments/18b0oal/questions-about-jsongson-serialization-and/?tl=it)
<https://stackoverflow.com/questions/12318097/why-do-we-use-serialization>
<https://stackoverflow.com/questions/633402/what-is-serialization>
<https://stackoverflow.com/questions/2232759/what-is-the-purpose-of-serialization-in-java>
<https://stackoverflow.com/questions/447898/what-is-object-serialization>
<https://stackoverflow.com/questions/16239239/why-does-the-defaultwriteobject-function-have-to-be-called-first-when-writing-in>
<https://stackoverflow.com/questions/285793/what-is-a-serialversionuid-and-why-should-i-use-it>
<https://stackoverflow.com/questions/12963445/serialization-readobject-writeobject-overrides>
<https://stackoverflow.com/questions/62882189/difference-between-search-maven-org-and-mvnrepository-com>
<https://stackoverflow.com/questions/8575516/creating-jar-file-of-maven-projects-with-netbeans>
<https://stackoverflow.com/questions/17318313/jackson-library-json-mapper-to-string>

Questioni di Sicurezza

<https://adtmag.com/blogs/watersworks/2018/06/java-serialization.aspx>

Creare una libreria .jar

<https://javatutorial.net/create-java-jar-file-with-maven/>

JSON e libreria Jackson

<https://www.json.org/json-it.html>
<https://github.com/FasterXML>
<https://mvnrepository.com/>
<https://jsonformatter.curiousconcept.com/>
<https://blogs.oracle.com/javamagazine/java-json-serialization-jackson/>
<https://www.baeldung.com/jackson-object-mapper-tutorial>
<https://en.wikipedia.org/wiki/JSON>
<http://www.davismol.net/category/programming/json/jackson/> ← è una lista di articoli dello stesso autore sul tema

XML

<https://it.wikipedia.org/wiki/XML>
<https://en.wikipedia.org/wiki/XML>
<http://www.davismol.net/2017/03/13/jaxb-marshalling-da-oggetto-java-a-xml/>

Differenze tra JSON e XML

<https://aws.amazon.com/it/compare/the-difference-between-json-xml/>
<https://www.json.org/xml.html>
<https://zuplo.com/learning-center/json-vs-xml-for-web-apis>
https://www.w3schools.com/js/js_json_xml.asp
<https://toolpix.pythonanywhere.com/blog/json-vs-xml>